

ETC4500/ETC5450

Advanced R programming

Week 7: Measuring and improving
performance



Outline

- 1 Profiling
- 2 Benchmarking
- 3 Vectorisation
- 4 Copy-on-modify
- 5 Efficient coding
- 6 Caching

Outline

- 1 Profiling
- 2 Benchmarking
- 3 Vectorisation
- 4 Copy-on-modify
- 5 Efficient coding
- 6 Caching

Profiling functions

- `Rprof()` : records every function call.
- `summaryRprof()` : summarises the results.
- `profvis()` : visualises the results.

Profiling

Where are the bottlenecks in your code?

```
f <- function() {  
  mean(rnorm(1e7))  
  g()  
  h()  
}  
  
g <- function() {  
  mean(rnorm(1e7))  
  h()  
}  
  
h <- function() {  
  mean(rnorm(1e7))  
}
```

Profiling

```
tmp <- tempfile()
Rprof(tmp, interval = 0.1)
f()
Rprof(NULL)
```

```
sample.interval=100000
"rnorm" "mean" "f"
"rnorm" "mean" "f"
"rnorm" "mean" "f"
"rnorm" "mean" "g" "f"
"rnorm" "mean" "g" "f"
"rnorm" "mean" "h" "g" "f"
"rnorm" "mean" "h" "g" "f"
```

```
"rnorm" "mean" "h" "f"
```

```
"mean.default" "mean" "h" "f"
```

Profiling

```
summaryRprof(tmp)
```

```
$by.self
```

	self.time	self.pct	total.time	total.pct
"rnorm"	0.8	88.89	0.8	88.89
"mean.default"	0.1	11.11	0.1	11.11

```
$by.total
```

	total.time	total.pct	self.time	self.pct
"f"	0.9	100.00	0.0	0.00
"mean"	0.9	100.00	0.0	0.00
"rnorm"	0.8	88.89	0.8	88.89
"g"	0.4	44.44	0.0	0.00
"h"	0.4	44.44	0.0	0.00


```
"mean.default"      0.1      11.11      0.1      11.11
```

```
$sample.interval
```

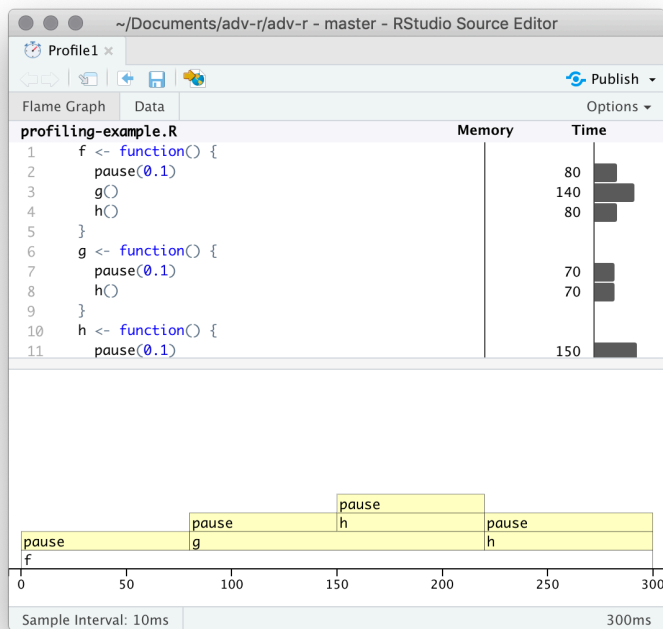
```
[1] 0.1
```

```
$sampling.time
```

```
[1] 0.9
```

Profiling

```
profvis::profvis(f())
```



Profiling

- Garbage collection: if it takes a long time, you're creating many short-lived objects.
- Can't see inside C/C++ code
- Anonymous functions make it harder to identify bottlenecks.
- Lazy evaluation means arguments are sometimes evaluated in a different function than where they are called.

Outline

- 1 Profiling
- 2 Benchmarking
- 3 Vectorisation
- 4 Copy-on-modify
- 5 Efficient coding
- 6 Caching

Microbenchmarking

`system.time()`

```
x <- rnorm(1e6)
system.time(min(x))
```

user	system	elapsed
0.001	0.000	0.001

```
system.time(sort(x)[1])
```

user	system	elapsed
0.028	0.004	0.032

```
system.time(x[order(x)[1]])
```

user	system	elapsed
0.021	0.004	0.024

Microbenchmarking

bench::mark()

```
bench::mark(  
  min(x),  
  sort(x)[1],  
  x[order(x)[1]]  
)
```

A tibble: 3 × 6

	expression	min	median	`itr/sec`	mem_alloc	`gc/sec`
	<bch:expr>	<bch:tm>	<bch:tm>	<dbl>	<bch:byt>	<dbl>
1	min(x)	795.3µs	803.7µs	1236.	0B	0
2	sort(x)[1]	27.4ms	29.2ms	34.2	11.44MB	18.7
3	x[order(x)[1]]	22ms	22.4ms	44.4	3.81MB	4.44

Microbenchmarking

- `mem_alloc` tells you the memory allocated in the first run.
- `n_gc` tells you the total number of garbage collections over all runs.
- `n_itr` tells you how many times the expression was evaluated.
- Pay attention to the units!

Exercises

2. What's the fastest way to compute a square root? Compare:

- `sqrt(x)`
- `x^0.5`
- `exp(log(x) / 2)`

Use `system.time()` find the time for each operation.

Repeat using `bench::mark()`. Why are they different?

Outline

- 1 Profiling
- 2 Benchmarking
- 3 Vectorisation**
- 4 Copy-on-modify
- 5 Efficient coding
- 6 Caching

Vectorisation

- Vectorisation is the process of converting a repeated operation into a vector operation.
- The loops in a vectorised function are implemented in C instead of R.
- Using `map()` or `apply()` is **not** vectorisation.
- Matrix operations are vectorised, and usually very fast.

Exercise

Write the following algorithm to estimate $\int_0^1 x^2 dx$ using vectorised code

Monte Carlo Integration

- a. Initialise: `hits = 0`
- b. for `i` in `1:N`
 - Generate two random numbers, U_1, U_2 , between 0 and 1
 - If $U_2 < U_1^2$, then `hits = hits + 1`
- c. end for
- d. Area estimate = `hits/N`

Exercise

Compare the speed of `apply(x, 1, sum)` and `rowSums(x)` for a large matrix `x`.

Outline

- 1 Profiling
- 2 Benchmarking
- 3 Vectorisation
- 4 Copy-on-modify**
- 5 Efficient coding
- 6 Caching

Copy-on-modify semantics

- R copies objects when modified — but only when necessary.
- Names bind to the same object until one is modified.
- Understanding this explains why many inefficiencies arise.

```
x <- c(1, 2, 3)
y <- x                                # no copy yet – x and y share memory
lobstr::obj_addr(x) == lobstr::obj_addr(y)
```

```
[1] TRUE
```

```
y[1] <- 10                            # copy happens now
lobstr::obj_addr(x) == lobstr::obj_addr(y)
```

```
[1] FALSE
```

Why growing objects is slow

Each `c(result, i)` copies the entire vector:

```
result <- c()  
for (i in 1:5) result <- c(result, i^2)
```

Pre-allocation avoids repeated copying:

```
result <- numeric(5)  
for (i in 1:5) result[i] <- i^2
```


Reference semantics

- Environments use **reference semantics** — modifications don't copy.
- `data.table` exploits this: `:=` modifies in place, no copy made.

```
library(data.table)
dt <- data.table(x = rnorm(1e6))
lobstr::obj_addr(dt)
```

```
[1] "0x5a0668489a80"
```

```
dt[, y := x^2]          # modifies dt directly
lobstr::obj_addr(dt)    # same address – no copy
```

```
[1] "0x5a0668489a80"
```

Reference semantics

Compare data.frame:

```
df <- data.frame(x = rnorm(1e6))  
lobstr::obj_addr(df)
```

```
[1] "0x5a066903c1a8"
```

```
df$y <- df$x^2 # modifies df, but creates a copy  
lobstr::obj_addr(df) # different address – copy made
```

```
[1] "0x5a066913d628"
```

Outline

- 1 Profiling
- 2 Benchmarking
- 3 Vectorisation
- 4 Copy-on-modify
- 5 Efficient coding
- 6 Caching

Other efficiency improvements

- Never calculate something more than once.
- Avoid growing objects in a loop. Allocate space first.
- Do as little as possible.
- Move code out of loops if possible.
- Parallelise if you can't vectorise.
- Use `data.table()` instead of `data.frame()` for large datasets.
- Use `matrix()` instead of `data.frame()` or `data.table()` if all data are the same type.
- Find a built in function that does what you want instead of writing your own.

Never calculate something more than once

```
x <- rnorm(1e4)
bench::mark(
  bad = c(mean = mean(x), var = var(x), sd = sqrt(var(x))),
  good = {
    v <- var(x)
    c(mean = mean(x), var = v, sd = sqrt(v))
  }
)
```

A tibble: 2 × 6

	expression	min	median	`itr/sec`	mem_alloc	`gc/sec`
	<bch:expr>	<bch:tm>	<bch:tm>	<dbl>	<bch:byt>	<dbl>
1	bad	60.6µs	63.5µs	15623.	15.3KB	4.55
2	good	37.8µs	39µs	25423.	0B	2.54

Avoid growing objects in a loop

```
n <- 1e4
bench::mark(
  growing = {
    result <- c()
    for (i in seq_len(n)) result <- c(result, i^2)
  },
  preallocated = {
    result <- numeric(n)
    for (i in seq_len(n)) result[i] <- i^2
  }
)
```

A tibble: 2 × 6

expression	min	median	`itr/sec`	mem_alloc	`gc/sec`
------------	-----	--------	-----------	-----------	----------

	<bch:expr>	<bch:tm>	<bch:tm>	<dbl>	<bch:byt>	<dbl>
1	growing	73.1ms	73.8ms	9.83	382MB	110.
2	preallocated	699.1μs	735μs	1251.	91.8KB	20.0

Do as little as possible

```
x <- rnorm(1e6)
bench::mark(
  slow = which(x == min(x))[1],
  fast = which.min(x)
)
```

```
# A tibble: 2 × 6
```

	expression	min	median	`itr/sec`	mem_alloc	`gc/sec`
	<bch:expr>	<bch:tm>	<bch:tm>	<dbl>	<bch:byt>	<dbl>
1	slow	1.81ms	1.91ms	508.	7.63MB	174.
2	fast	377.66µs	384.73µs	2599.	2.09KB	0

Move code out of loops

```
n <- 1e4
x <- rnorm(n)
bench::mark(
  inside = {
    result <- numeric(n)
    for (i in seq_len(n)) result[i] <- x[i] * mean(x)
  },
  outside = {
    mx <- mean(x)
    result <- numeric(n)
    for (i in seq_len(n)) result[i] <- x[i] * mx
  }
)
```

```
# A tibble: 2 × 6
```

	expression	min	median	`itr/sec`	mem_alloc	`gc/sec`
	<bch:expr>	<bch:tm>	<bch:tm>	<dbl>	<bch:byt>	<dbl>
1	inside	145ms	145ms	6.87	97.4KB	6.87
2	outside	827µs	872µs	1129.	94.4KB	22.9

Parallelise if you can't vectorise

```
# Sequential
purrr::map(1:8, slow_fn)

# Parallel
future::plan(multisession, workers = 4)
furrr::future_map(1:8, slow_fn)
```

Use `data.table` for large datasets

```
library(data.table)
library(dplyr)
n <- 1e6
df <- data.frame(g = sample(letters, n, replace = TRUE), x = rnorm(n))
dt <- as.data.table(df)
bench::mark(
  data.frame = summarise(df, mean(x), .by = g),
  data.table = dt[, .(mean = mean(x)), by = g],
  check = FALSE
)
```

```
# A tibble: 2 × 6
```

expression	min	median	`itr/sec`	mem_alloc	`gc/sec`
<bch:expr>	<bch:tm>	<bch:tm>	<dbl>	<bch:byt>	<dbl>

1	data.frame	11.1ms	11.47ms	87.2	27.3MB	209.
2	data.table	4.85ms	6.41ms	153.	27.6MB	218.

Use `matrix` for homogeneous data

```
n <- 500
df <- as.data.frame(matrix(rnorm(n^2), nrow = n))
mat <- as.matrix(df)
bench::mark(
  data.frame = t(df) %*% as.matrix(df),
  matrix = t(mat) %*% mat
)
```

```
# A tibble: 2 × 6
```

	expression	min	median	`itr/sec`	mem_alloc	`gc/sec`
	<bch:expr>	<bch:tm>	<bch:tm>	<dbl>	<bch:byt>	<dbl>
1	data.frame	2.97ms	3.11ms	263.	7.67MB	37.6
2	matrix	989.68µs	1.03ms	661.	3.81MB	39.6

Outline

- 1 Profiling
- 2 Benchmarking
- 3 Vectorisation
- 4 Copy-on-modify
- 5 Efficient coding
- 6 Caching

Caching: using rds

```
if (file.exists("results.rds")) {  
  res <- readRDS("results.rds")  
} else {  
  res <- compute_it() # a time-consuming function  
  saveRDS(res, "results.rds")  
}
```

```
res <- xfun::cache_rds(  
  compute_it(), # a time-consuming function  
  file = "results.rds"  
)
```


Caching: using rds

```
compute <- function(...) {  
  xfun::cache_rds(rnorm(6), file = "results.rds", ...)  
}  
compute()
```

```
[1]  1.4642 -0.5088  0.9769 -1.9678  1.2052  0.0603
```

```
compute()
```

```
[1]  1.4642 -0.5088  0.9769 -1.9678  1.2052  0.0603
```

```
compute(rerun = TRUE)
```

```
[1]  0.262 -1.338 -0.963  0.833 -0.457  2.370
```

```
compute()
```

```
[1] 0.262 -1.338 -0.963 0.833 -0.457 2.370
```

Caching downloads

Prevent downloads of the same data multiple times.

```
download_data <- function(url) {  
  dest_folder <- tempdir()  
  sanitized_url <- stringr::str_replace_all(url, "/", "_")  
  dest_file <- file.path(dest_folder, paste0(sanitized_url, ".rds"))  
  if (file.exists(dest_file)) {  
    data <- readRDS(dest_file)  
  } else {  
    data <- readr::read_tsv(url, show_col_types = FALSE)  
    saveRDS(data, dest_file)  
  }  
  data  
}
```

```
bulldozers <- download_data(  
  "https://robjhyndman.com/data/Bulldozers.csv"  
)
```

Caching: memoise

Caching stores results of computations so they can be reused.

```
library(memoise)
sq <- function(x) {
  cat("Computing square of 'x'")
  x^2
}
memo_sq <- memoise(sq)
memo_sq(2)
```

```
Computing square of 'x'
```

```
[1] 4
```

```
memo_sq(2)
```

```
[1] 4
```

Exercises

Use `bench::mark()` to compare the speed of `sq()` and `memo_sq()`.

Caching: Quarto

```
```{r}  
#| label: import-data
#| cache: true
d <- read.csv('my-precious.csv')
```
```

```
```{r}  
#| label: analysis
#| dependson: import-data
#| cache: true
summary(d)
```
```


Caching: Quarto

- Requires explicit dependencies or changes not detected.
- Changes to functions or packages not detected.
- Good practice to frequently clear cache to avoid problems.
- `targets` is a better solution